

Multiplayer Game Platform using Java RMI

Project Overview

This project requires you to develop a distributed multiplayer game (XO for example) platform using Java Remote Method Invocation (RMI). The system will enable multiple players to connect to a central server, view other connected players, check scores, initiate game sessions, and play interactive games in real-time with other players.

Note: This project focuses on mastering distributed systems concepts (RMI, synchronization, concurrency) rather than game complexity.

Core Requirements

1. RMI Architecture: Implement a client-server architecture using Java RMI where:

- The server hosts game services and manages player connections
- Clients connect to the server to access game functionalities

2. Player Management:

- Allow new players to register with username/ID
- Maintain a list of currently connected players
- Handle player disconnections gracefully
- Display player status (online/busy/offline)

3. Game Invitation System:

- Enable players to view the list of available players
- Allow players to send game invitations to others
- Implement invitation acceptance/rejection functionality

4. Gameplay Implementation:

- Synchronize game state between players in real-time
- Enforce game rules and validate moves
- Track game progress and determine winners
- All game logic and validation must be handled by the server.
- Games should be turn-based to simplify synchronization.

5. **Score Tracking:**

- Record win/loss statistics for each player
- Implement a leaderboard system
- Persist player scores between application sessions using **file** storage

6. **Exception Handling:**

- Implement robust error handling for network issues
- Handle unexpected player disconnections during gameplay (optional)
- Provide appropriate feedback for error scenarios

Technical Requirements

1. **Java RMI Implementation:**

- Define and implement remote interfaces for all distributed services
- Register services with the RMI registry
- Implement appropriate callback mechanisms for real-time updates

2. **Concurrency Control:**

- Handle multiple simultaneous game sessions
- Ensure thread safety for shared resources
- Manage concurrent access to player data

3. **Data Persistence:**

- Store player information and scores in files
- Implement data loading/saving functionality
- Ensure data integrity during concurrent operations

4. **User Interface:**

- Create a basic but functional user interface for player interaction
- Display game state, player list, and scores clearly
- Provide intuitive controls for gameplay

Alternative Game Suggestions

Instead of the traditional XO (Tic-Tac-Toe) game, students may implement one of the following games:

1. **Word Guessing Game:** Players take turns guessing a hidden word with limited attempts. The server provides clues after each guess.
2. **Connect Four:** Players drop colored discs into a vertical grid, aiming to connect four of their discs in a row.
3. **Battleship:** Each player positions ships on a grid and takes turns guessing coordinates to sink the opponent's ships.
4. **Dots and Boxes:** Players take turns connecting dots on a grid. When a player completes a box, they claim it and get another turn.
5. **Rock Paper Scissors:** A simple game with extended features like score tracking and multiple rounds.
6. **Memory Match:** Players take turns flipping pairs of cards to find matches, with the goal of finding the most pairs.
7. **Hangman:** One player thinks of a word, and the other tries to guess it by suggesting letters within a limited number of guesses.
8. **Snakes and Ladders Game:** Players roll dice to advance on a board, with snakes and ladders affecting their progress.
9. **Checkers:** A simplified version where players move pieces diagonally to capture opponent pieces.
10. **Word Chain:** Each player must provide a word that begins with the last letter of the previous player's word.

Advanced Features (Optional)

1. **Multiplayer Games:** Support games with more than two players simultaneously
2. **Chat System:** Implement a chat feature for players to communicate during gameplay
3. **Game Replay:** Allow recording and replaying of past games
4. **Custom Game Settings:** Let players customize game parameters before starting
5. **Reconnection Mechanism:** Allow players to reconnect and continue their game after disconnection

6. **Multiple Game Support:** Implement several different games with a unified lobby system

Evaluation Criteria

1. **RMI Implementation:** Correct usage of RMI concepts and proper distributed communication
2. **Functionality:** Working gameplay with proper rule enforcement
3. **Robustness:** Handling of exceptions and edge cases
4. **Code Quality:** Well-structured, documented, and maintainable code
5. **User Experience:** Intuitive interface and smooth gameplay

Submission Requirements

1. Source code with documentation
2. User manual explaining how to run the application
3. Technical report describing the architecture and implementation details
4. Brief presentation demonstrating the application's functionality
5. Live Demonstration: Each group must present a live demo of their application. The demo should show:
 - Two or more clients connecting to the server
 - Player interaction (invitation, acceptance, gameplay)
 - A complete game session until a result is obtained
 - Basic error handling (e.g., player disconnection or invalid move)

This project will help students understand distributed application development, network programming, concurrency control, and interactive application design using Java RMI technology.